

Chirix ERP integration

Setup guide for the Navin Plastic Tech CRM

Sales orders raised in Chirix appear in this system automatically, so the plant is not typing them a second time [BLUEPRINT §12].

This guide is the practical half: how to get the credential, where to put it, what the integration sends, what it expects back, and what happens when the same order arrives twice — which it will, on every poll, by design.

Read CHIRIX_API_REQUEST.md first if the API details are not yet in hand

That document is the list of questions to put to the vendor, and why each one matters. Several decisions below depend on answers only Chirix can give; each is marked where it comes up.

Status: everything but the HTTP client is built

The matching, the de-duplication and the amendment rules are written and tested. The client that actually calls Chirix is deliberately last, because writing a normaliser against a guessed response shape is how a field mapping goes silently wrong. Section 4 says exactly what it has to produce.

1 · Getting the API key

Ask Chirix for a **read-only** credential scoped to sales orders. The integration never writes back, so a credential that *can* write is a standing risk with no upside — say so when you ask, because a vendor's default is usually a full-access key.

What they give you	What it means	What to set
An API key or token	The usual case	CHIRIX_API_KEY, sent as Authorization: Bearer <key>
A key for a header of their own	e.g. X-API-Key: <key>	CHIRIX_API_KEY plus CHIRIX_AUTH_HEADER=X-API-Key and CHIRIX_AUTH_SCHEME= (empty)
A username and password	Basic auth	Base64 user:pass into CHIRIX_API_KEY, CHIRIX_AUTH_SCHEME=Basic
OAuth client credentials	A token has to be fetched and refreshed	Not covered by these settings — the adapter needs a token step. Tell them we would prefer a static key

Ask at the same time for a **test environment**. If there is none, ask for one real sales order as their API returns it, with the customer name and prices replaced by dummy values. The field names are what we need, not the data — and a real response shows what is actually there, including the fields that are always empty, which documentation never does.

Keeping it safe

The key reads every sales order in the business: what every customer buys, at what price. Treat it exactly like a password.

- It goes in `.env` on the server, which is git-ignored. **Never commit it** — not in a test fixture, not in a log, not in a screenshot pasted into a chat.
- Keep it out of the repository's example file. `.env.example` ships the *names* of the settings and no values, and that is the pattern to follow.
- If it is ever exposed, ask Chirix to revoke and re-issue rather than hoping. A read-only key is still a full copy of the order book.

2 · Configuring this system

Every setting lives in `.env` on the server. The feed is **off** until `CHIRIX_API_KEY` has a value — no polling, no warnings, nothing in the log. That is the correct state for a deployment that does not run Chirix, and the correct state for this one until the vendor has answered.

```
# Required — the feed is off without these two.
CHIRIX_API_KEY=
CHIRIX_API_URL=https://erp.chirix.example/api/v2

# How the key is presented. The defaults suit most vendors.
CHIRIX_AUTH_HEADER=Authorization
CHIRIX_AUTH_SCHEME=Bearer

# Polling.
CHIRIX_POLL_MINUTES=15
CHIRIX_BACKFILL_DAYS=7
CHIRIX_OVERLAP_MINUTES=30
CHIRIX_TIMEOUT_MS=20000

# Who an imported order belongs to when nothing else resolves an owner.
CHIRIX_FALLBACK_OWNER_EMAIL=nandhini@npthangers.com
```

Restart after editing

Nothing re-reads `.env` live. Run `pm2 restart npt-api`.

Why the poll is every fifteen minutes and not every one

A sales order is not a lead. Nothing downstream of one happens in under an hour — the \$13 verification checks alone take longer than that — so polling harder buys nothing and spends the vendor's rate limit. Ask Chirix what their limit actually is (question 5 in the request document) and set this **above** it, not at it.

Nothing is lost by polling slowly. The watermark means a slow poll is late, never incomplete.

Why the overlap exists

Each poll asks for orders modified since the last one. That timestamp is **Chirix's clock, not ours**, and two servers' clocks disagree by seconds at best. An order saved a moment either side of the watermark would fall between two windows and never arrive at all.

So every poll reaches back `CHIRIX_OVERLAP_MINUTES` further than it strictly needs to, and deliberately re-reads orders it has already seen. **This is free**, because of section 3 — and it is the reason section 3 had to be built before this one.

The fallback owner

§29 requires every customer, and every order, to belong to one marketing person. An imported order resolves its owner in three steps: the Chirix salesperson through a mapping the plant maintains, then the buyer's existing owner, then this fallback. Set it to a real, active person — an order that cannot be given an owner is refused and counted as a failure, which is loud but is still an order sitting outside the order book.

3 · What happens when the same order arrives twice

It does, constantly, and that is the design rather than a fault:

- the overlap above re-reads the last half hour on every poll;
- a request that times out here may have succeeded there, so it is retried;
- a restart resets the cursor;
- and somebody may have typed the order in by hand before the poll caught up.

Every one of these produces nothing. A sales order carries the pair `externalRef.source + externalRef.id` — chirix and their identifier for the order — and there is a **unique index** on that pair in the database.

Not a check in application code: a `findOne` followed by a `create` is two statements with a gap in the middle that two overlapping polls will both walk through. The importer does a single keyed upsert, so the loser of that race loses at the storage layer, where losing is safe.

Outcome	What it means
created	New. A sales order was raised, numbered SO-..., at <i>PO received</i>
unchanged	Seen before and nothing has moved. The common case
amended	Seen before, changed, and the plant had not started — applied
queried	Seen before, changed, and the plant <i>had</i> started — not applied; a question was raised
failed	The row could not be imported. Counted, reported, and the batch carried on

Typing one in by hand

If an order is phoned through and entered here before the poll fetches it, put the Chirix identifier on it — the booking form has a panel, “*Is this order already in another system?*”. The poll then recognises it and updates in place instead of booking a second one.

Without that, the poll finds nothing carrying that reference and creates a duplicate — exactly the failure the reference exists to prevent, arriving through the gap between the two ways an order can be entered.

Amendments: the rule that matters most

When Chirix changes an order we already have, what happens depends on whether the plant has started.

When	What happens
Before release PO received, verifying, clarification pending	The change is applied, and the §13 checks are cleared . They were ticked against figures that have just moved, and a “correct model” tick from before the model changed is worse than no tick — it reads as though somebody checked.
After release anything the plant has started	The change is never applied . The order stays exactly as the plant is running it, and an urgent question goes to marketing naming both sides: “ <i>Chirix has amended SO-1042 since this order was released: NPT-400S quantity 20,000 → 5,000.</i> ” A person decides.

Why an amendment is never applied to a running order

A quantity quietly rewritten under a running press is how the wrong quantity gets made. This is the failure the whole design exists to avoid, and it is the one rule here worth arguing about before changing.

An unanswered amendment is **not** re-asked on every poll — the revision is recorded even though nothing else is, so the question is raised once rather than every fifteen minutes until somebody answers it.

One bad row does not stop the batch

Twenty orders arrive and one names a buyer with a malformed GSTIN: the other nineteen still come in, and the bad one is reported with its identifier and a reason. An import that refuses a whole batch over one row is one people stop trusting and start double-checking by hand — at which point it has saved nothing.

4 · Writing the adapter

The only piece left. Everything below the adapter is built and tested; the adapter's whole job is to turn Chirix's response into the row shape the importer already accepts.

Request

```
GET {CHIRIX_API_URL}/sales-orders
    ?modifiedSince=<ISO-8601>&page=<n>&pageSize=<n>

Authorization: Bearer <CHIRIX_API_KEY>
Accept: application/json
```

Three of these are guesses until Chirix answers:

- **modifiedSince** — the parameter name, and whether the timestamp is IST or UTC (question 3). Send it in whatever zone they name, explicitly offset.
- **Paging** — page number, offset or cursor, and the maximum page size (question 4). Walk it to the end and prove you reached it.
- **Order lines** — whether they come in the list response or need a second call per order (question 6). If they need one, a hundred orders is a hundred and one requests, which runs straight into their rate limit.

The timezone half is not pedantry

An IST timestamp read as UTC skips five and a half hours of orders, silently, and only in that window. It is the classic integration bug that surfaces a month later as “some orders don't come through”.

A list endpoint that silently returns only the first 50

...is how an integration looks like it works and quietly drops the rest. Walk the pages to the end, and assert you got there.

Response

Whatever shape they send, the adapter normalises each order into this, which is what `importBatch` takes:

```
{
  // Required. Their id for the order – what de-duplication turns on.
  externalId: 'SO-1042',

  // Their revision, when they have one. Ask for it (question 7):
  // without it, 'changed' has to be inferred by comparing fields.
  externalRef: { revision: '3' },

  // The buyer. GSTIN matched first, then name, then name+mobile.
  // Nothing is required – an unknown buyer is created and flagged.
  customer: {
    gstin: '33AABCS1429B1ZP',
    name: 'Sri Kumaran Knits Pvt Ltd',
    mobile: '9840011223',
    city: 'Tiruppur',
    state: 'Tamil Nadu',
  },

  // Their salesperson, mapped to one of our users through a table.
  // Never matched fuzzily: 'R. Kumar' vs 'Ramesh Kumar' is the guess
  // this refuses everywhere.
  salesperson: 'Nandhini',

  customerPo: { number: 'PO/2026/88', date: '2026-09-01' },
  orderDate: '2026-09-01',

  gstPercent: 18,
  isExport: false,
  paymentTerms: '30 days from invoice',
  deliveryTerms: '4 weeks',
  freightTerms: 'ex_factory',

  // At least one. mouldCode matched exactly against the register,
  // then modelNumber – and nothing else.
  lines: [{
    mouldCode: 'M-NH-400',
    modelNumber: 'NPT-400S',
    colour: 'White',
```

```

    printing: '1 COLOUR',
    packing: '200 per carton',
    quantity: 20000,    // required, a positive integer
    unitPrice: 7.5,     // required
    deliveryDate: '2026-10-15',
  }],
}

```

Why the mould is matched exactly and never approximately

A wrong customer is embarrassing and correctable. A wrong tool is fifty thousand pieces on the wrong steel. Where a code does not match exactly the answer is “unmatched”, recorded in words for order confirmation to read — never “probably this one”.

Then:

```

import { importBatch, summarise } from
  './services/orderImport.service.js';

const result = await importBatch(rows, {
  source: 'chirix',
  mapping: { Nandhini: nandhiniUserId }, // theirs -> ours
  fallback: fallbackUserId, // CHIRIX_FALLBACK_OWNER_EMAIL
  by: importUserId, // who the audit trail records
});

console.log(summarise(result));
// imported 3 new, 1 amended, 0 queried, 47 unchanged, 0 failed

```

Field mapping notes

- **Dates.** Parse to a real Date in the adapter, not downstream. A string that reaches the model is cast using the server's zone, which is not necessarily theirs.
- **Money.** Their unit price may include tax or exclude it. Ask which — an 18% error in every imported rate is not something anybody notices until a margin report looks wrong.
- **Cancellations.** A status field, not the order vanishing from the feed. Absence is not cancellation: a page boundary, a filter change or a slow query all look identical to a deletion, and acting on that inference would close a live order.
- **Fields we do not carry** are ignored, which is fine — except that it makes “has this changed?” less reliable when there is no revision number. One more reason to ask for one.

5 · Checking it works

Once the adapter exists:

- 1 **Run it once against the test environment**, if there is one, and read the summary line. Every count should be created on the first run.
- 2 **Run it again immediately.** Every count should be unchanged, and nothing new should appear in the order register. **This is the test that matters** — if anything is created on the second run, the identifier is not stable and everything else is built on sand.

- 3 **Change one order in Chirix and run again.** It should come back amended, and its §13 checks should be clear.
- 4 **Release an order here, change it in Chirix, run again.** It should come back queried, the order should be untouched, and an urgent question should be sitting in marketing's queue.

The automated equivalents of all four live in `tests/order-import.test.js` and run with the suite.

6 · What this integration deliberately does not do

- **Write to Chirix.** One direction, always. Two systems writing to each other produce a class of disagreement that cannot be debugged from either side, and nothing in the plan needs it.
- **Connect to their database.** Faster to build, and it couples us to their schema so it breaks silently whenever they upgrade. Through the API we break loudly, which is the failure we want.
- **Match anything fuzzily.** Not the buyer, and emphatically not the mould.
- **Skip the §13 gate.** An imported order goes through exactly the same eight checks as one typed here. It arrives faster; it does not arrive more trusted.

Navin Plastic Tech · CRM/ERP · Chirix ERP integration · companion to docs/CHIRIX_SETUP.md and docs/CHIRIX_API_REQUEST.md